

LAYERZERO - SECURITY REVIEW

Solana OApp.

A security review of LayerZero Solana OApp contracts, macros, RBAC utilities, and Anchor trait framework code.

CLIENT

LayerZero

PROJECT

Solana OApp

AUDITORS

Ali Shehab -
Osman Ozdemir -
Zdravko Hristov

AUDIT WINDOW

April 10 - April 30,
2026

FINAL REPORT

July 3, 2026

§ 00

Contents.

67 pages - overview, confidence ranking, scope, 54 findings, disclaimer, and about Guardian.

§ 01	Overview	p. 03
§ 02	Confidence Ranking	p. 04
§ 03	Scope & Classifications	p. 05
	Files in Scope	p. 06
§ 04	Findings	p. 07
	R1-M-01 to R1-M-01	p. 26-26
	Low · R1-L-01 to R2-L-01	p. 13-52
	Info · R1-I-02 to R4-I-02	p. 12-65
§ 05	Disclaimer	p. 66
§ 06	About Guardian	p. 67

Overview.

Guardian reviewed LayerZero's Solana OApp over two main-review tables and two remediation-review tables. The final report includes one Medium, nine Lows, and forty-four Informationals.

AUDIT FIRM	Guardian	REPOSITORIES	LayerZero-Labs/audit-external
CLIENT	LayerZero	METHODOLOGY	Static - Manual - Test
PROJECT	Solana OApp	MAIN REVIEW COMMIT	983d30d50b3a27b4b91a6213fbc649055e1a6d15
PREPARED BY	Ali Shehab - Osman Ozdemir - Zdravko Hristov	REMEDIATION COMMIT	f714feccb769986cb325d6c14bc89993a8fbba08
AUDIT WINDOW	April 10 - April 30, 2026		
FINAL REPORT	July 3, 2026		

1.1 Confidence Ranking

5/5

Guardian assigns a Confidence Ranking of 5 out of 5. No Critical or High findings were identified, and the review surfaced one Medium issue, Low-severity validation/documentation concerns, and Informational items. Findings are resolved or acknowledged following remediation review.

Very High Confidence

1 V. LOW	2 LOW	3 MED	4 HIGH	5 V. HIGH
-------------	----------	----------	-----------	--------------

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	IN PROG	DECLINED	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0	0	0
High	0	0	0	0	0	0	0
Medium	1	0	0	0	0	0	1
Low	9	0	0	1	2	0	6
Info	44	0	0	0	8	0	36

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3 Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas such as weak modularity or risky patterns. No critical design flaws were identified, though some patterns could lead to edge-case issues. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2 Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week, or additional unresolved High/Critical findings uncovered in remediation review.
1 Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

Scope & Classifications.

The audit covered LayerZero Solana OApp source, OApp macro generation, RBAC utilities, and the shared Anchor trait framework. Scope totals are grouped from the Notion export cloc output.

3.1 Severity Matrix

	IMP · HIGH	IMP · MEDIUM	IMP · LOW	IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
LIK · HIGH	Critical	High	Medium	IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
LIK · MED	High	Medium	Low	IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIK · LOW	Medium	Low	Low	LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
				LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
				LIKELIHOOD · LOW	Unlikely to ever occur in production.

§ 03

Scope.

Files in scope, grouped by repository. SLOC counts are physical source lines.

LayerZero-Labs/audit-external

54 CONTRACTS · 2,670 SLOC

apps/oapp-app/contracts/solana/src/ area total	755
apps/oapp-app/contracts/solana/macros/src/ area total	665
contracts/common/framework/solana/anchor-trait/src/ area total	407
contracts/common/utils/solana/rbac/macros/src/ area total	729
contracts/common/utils/solana/rbac/src/ area total	114

§ 04

Findings.

Fifty-four findings across four review tables: one Medium, nine Lows, and forty-four Informationals.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
R1-I-02	Override can desync OAppInfo and SDK discovery	Validation	INFO	RESOLVED	p. 12
R1-L-01	Payer-funder mismatch in enforced options	Logical Error	LOW	RESOLVED	p. 13
R1-L-02	Admin transfer reverts if OApp not registered	Validation	LOW	RESOLVED	p. 14
R1-L-03	Admin transfer uses unvalidated accounts	Validation	LOW	DECLINED	p. 15
R1-I-01	EnforcedOptions space vs field order mismatch	Informational	INFO	RESOLVED	p. 16
R1-L-04	Missing #[repr(u8)] enable role seed collision	Validation	LOW	RESOLVED	p. 17
R1-L-05	only_role Does Not Enforce Signer	Validation	LOW	RESOLVED	p. 18
R1-I-03	Misleading set_delegate mention in macro doc	Informational	INFO	RESOLVED	p. 19
R1-I-04	Iz_receive_types_info accounts undocumented	Best Practices	INFO	RESOLVED	p. 20
R1-I-05	Unused error variant OAppError::InvalidPeer	Informational	INFO	RESOLVED	p. 21
R1-I-06	RBAC role-admin mapping is fixed at compile time	Informational	INFO	ACKNOWLEDGED	p. 22
R1-L-06	Fully-qualified attribute paths not matched	Validation	LOW	RESOLVED	p. 23

§ 04

Findings, continued.

Index page 2 of 5.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
R1-I-07	Marker attribute macro bodies are unreachable	Informational	INFO	RESOLVED	p. 24
R1-I-08	Duplicate use super::* in generated module	Informational	INFO	ACKNOWLEDGED	p. 25
R1-M-01	OApp macro rejects module inner attributes	Logical Error	MEDIUM	RESOLVED	p. 26
R1-I-09	endpoint_cpi Helpers Can Panic on Short Slices	Validation	INFO	RESOLVED	p. 27
R1-L-07	Override Handler Cannot Access Nested Module Helpers	Logical Error	LOW	RESOLVED	p. 28
R1-I-10	Events omit the state instance pubkey	Events	INFO	RESOLVED	p. 29
R1-I-11	The default_admin_role should not be dynamic	Warning	INFO	RESOLVED	p. 30
R1-L-08	Peer/options accounts cannot be closed to reclaim rent	Best Practices	LOW	ACKNOWLEDGED	p. 31
R1-I-12	RoleRevoked.sender doc copy-pasted from grant	Informational	INFO	RESOLVED	p. 32
R1-I-13	Clarify endpoint program check in CPI wrappers	Documentation	INFO	RESOLVED	p. 33
R1-I-14	Misleading Code Comment	Documentation	INFO	RESOLVED	p. 34
R1-I-15	Endpoint CPI wrappers rely on account ordering	Suggestion	INFO	RESOLVED	p. 35

§ 04

Findings, continued.

Index page 3 of 5.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
R1-I-16	OApp initialize() is frontrun-hijackable	Frontrunning	INFO	ACKNOWLEDGED	p. 36
R1-I-17	RBAC docs reference non-existent # [grant_role]	Documentation	INFO	RESOLVED	p. 37
R1-I-18	deserialize_alt Can Panic Via unwrap	Informational	INFO	RESOLVED	p. 38
R1-I-19	Hardcoded crate path in declare_instructions	Compatibility	INFO	RESOLVED	p. 39
R1-I-20	Arbitrary marker accepted instead of (ctx)	Compatibility	INFO	RESOLVED	p. 40
R1-I-21	Instruction aliases emit duplicate entrypoints	Documentation	INFO	RESOLVED	p. 41
R1-I-22	Docs understate default impl remains available	Documentation	INFO	RESOLVED	p. 42
R1-I-23	Default accounts generics are not supported	Compatibility	INFO	RESOLVED	p. 43
R1-I-24	Field-Injection Macros Always Append New Fields	Documentation	INFO	RESOLVED	p. 44
R1-I-25	OAppInfo can be silently uninitialized	Compatibility	INFO	RESOLVED	p. 45
R1-I-26	Generated Code Is Susceptible To Name Shadowing	Compatibility	INFO	RESOLVED	p. 46
R2-I-01	Unconstrained sender in setup_default_admin!	Events	INFO	RESOLVED	p. 47

§ 04

Findings, continued.

Index page 4 of 5.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
R2-I-02	clear wrapper docs imply OApp-only signer	Validation	INFO	RESOLVED	p. 48
R2-I-03	Stale OAppInfo Documentation After TLV Migration	Documentation	INFO	RESOLVED	p. 49
R2-I-04	anchor-trait README stale on wrapper_defs	Documentation	INFO	RESOLVED	p. 50
R2-I-05	Consider Size Check In update_tlv_entry	Validation	INFO	RESOLVED	p. 51
R2-L-01	OAppInfo Can Reference Missing TLV Entry	Validation	LOW	ACKNOWLEDGED	p. 52
R2-I-06	Incorrect composer docs link	Documentation	INFO	RESOLVED	p. 53
R2-I-07	Private repo link in source comment	Documentation	INFO	RESOLVED	p. 54
R2-I-08	Incomplete package files list	Configuration	INFO	RESOLVED	p. 55
R2-I-09	register_oapp Prerequisite Documentation Gap	Documentation	INFO	RESOLVED	p. 56
R2-I-10	OAppInfo init accepts nonzero TLV tail	Validation	INFO	RESOLVED	p. 57
R2-I-11	Program attr filtering should be documented	Documentation	INFO	RESOLVED	p. 58
R2-I-12	RoleType derive can use nonzero default	Validation	INFO	RESOLVED	p. 59

§ 04

Findings, continued.

Index page 5 of 5.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
R3-I-01	Composer codegen doc points to the wrong file	Documentation	INFO	ACKNOWLEDGED	p. 60
R3-I-02	Typo in CallerNotPendingAdmin doc comment	Documentation	INFO	ACKNOWLEDGED	p. 61
R3-I-03	Broken intra-doc link to CodegenContext	Best Practices	INFO	ACKNOWLEDGED	p. 62
R3-I-04	Misleading clear documentation	Documentation	INFO	ACKNOWLEDGED	p. 63
R4-I-01	Stale OAppBase doc snippet in oapp-info	Documentation	INFO	ACKNOWLEDGED	p. 64
R4-I-02	OAppInfo v1 Can Allow Misleading Discovery	Documentation	INFO	RESOLVED	p. 65

INFO R1-I-02

RESOLVED

Override can desync OAppInfo and SDK discovery.

CATEGORY Validation

LOCATION contracts/common/utils/solana/rbac/macros/src/lib.rs#L56-L59

REPO Round 1 - Main Review

DESCRIPTION

The `#[oapp_instruction] / #[rbac_instruction]` override model accepts arbitrary `Context<T>`, but `OAppInfo.instructions.*` stays at the default standard version (for example `1`) unless the developer explicitly marks that instruction as custom during `init_oapp_info!`.

In `oapp_info`, instruction version bytes are SDK discovery signals: `0` means custom discovery (SDK falls back to `ExtraAccountMetaList`), while `>0` means standard discovery at that version (SDK uses predefined `Std*` account templates).

This creates a silent app-to-SDK contract mismatch:

- On-chain: instruction account layout has changed (custom `Context<T>`).
- Off-chain: SDK still reads version `1` and builds the predefined `Std*` account template.

As a result, callers may see either:

- functional breakage (account deserialization/position mismatch and revert), or
- unexpected success with unintended semantics when extra accounts are ignored.

The framework does not enforce or warn that custom overrides must be paired with `init_oapp_info!(..., [InstructionName])` (version `0`, custom discovery).

RECOMMENDATION

Consider adding an explicit consistency guard between override collection and codegen; when an instruction is overridden with non-`Std*` accounts, require it to be marked custom in `init_oapp_info!` (or fail/warn at compile time).

RESOLUTION

LayerZero Team - Resolved.

LOW **R1-L-01**

RESOLVED

Payer-funder mismatch in enforced options.

CATEGORY Logical Error

LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/set_enforced_options.rs#L51

REPO Round 1 - Main Review

DESCRIPTION

The `payer` in `StdSetEnforcedOptions` (`set_enforced_options.rs:44`) is an unconstrained `Signer<'info>`, independent of the account that originally funded the `EnforcedOptions` PDA in `init_enforced_options`. Both instructions declare `payer` as a separate field from `authority`, with no constraint tying them together or tracking who originally funded the PDA. When the options buffer shrinks during a `set_enforced_options` call, Anchor's `realloc::payer = payer` directive routes the reclaimed rent lamports to the current transaction's `payer` - not the account that originally paid the rent in `init_enforced_options`. This creates a funder/refund-recipient mismatch: account A funds the PDA at initialization, but account B receives the refund when the PDA shrinks.

RECOMMENDATION

Consider separating the `payer` (who funds growth) from a `refunder` (who receives lamports on shrink) and handling the refunds manually. Alternatively, consider documenting this behavior explicitly so integrators are aware that rent refunds go to the current `payer`, not the original funder.

RESOLUTION

LayerZero Team - Resolved.

LOW **R1-L-02**

RESOLVED

Admin transfer reverts if OApp not registered.

CATEGORY Validation

LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/accept_default_admin_transfer.rs#L25-L33

REPO Round 1 - Main Review

DESCRIPTION

The `#[oapp]` macro unconditionally injects a `set_delegate` CPI call into the `accept_default_admin_transfer` handler (`accept_default_admin_transfer.rs`). This CPI targets the endpoint's `SetDelegate` instruction, which requires the `OAppRegistry` PDA (seeds `[OAPP_SEED, oapp.key]`) to exist and be initialized. The `OAppRegistry` is only created when the developer calls `endpoint_cpi::register_oapp` during their custom `initialize` instruction; a step the `#[oapp]` macro neither generates nor enforces.

If a developer deploys an OApp using the `#[oapp]` macro but omits the `register_oapp` CPI in their initialization handler, the RBAC system works normally; `setup_default_admin!` sets the admin, roles can be granted, and `begin_default_admin_transfer` succeeds since it only modifies program state. However, when the pending admin calls `accept_default_admin_transfer` , the RBAC portion (`StdAcceptDefaultAdminTransfer::apply`) completes but the subsequent `set_delegate` CPI fails because the `OAppRegistry` PDA does not exist.

An OApp deployed without calling `register_oapp` during initialization has a permanently bricked admin transfer flow.

RECOMMENDATION

Consider adding a guard in the injected `accept_default_admin_transfer` override that checks whether the `OAppRegistry` PDA exists before attempting the `set_delegate` CPI. If the registry does not exist, either skip the CPI (with a warning event) or return a descriptive error. Alternatively, document that `register_oapp` is a mandatory step during initialization.

RESOLUTION

LayerZero Team - Resolved.

LOW **R1-L-03**

DECLINED

Admin transfer uses unvalidated accounts.

CATEGORY Validation

LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/accept_default_admin_transfer.rs#L28

REPO Round 1 - Main Review

DESCRIPTION

Every endpoint CPI in the OApp framework uses static, program-controlled accounts. During `initialize`, the developer hardcodes the endpoint accounts for `register_oapp` and `set_delegate` directly in their Anchor accounts struct; the caller cannot influence which accounts reach the CPI. The same applies to `send`, `clear`, `send_compose`, and `clear_compose`: the OApp's instruction context declares all required endpoint accounts as named fields, so account ordering and correctness are enforced by Anchor constraints at deserialization time.

The `accept_default_admin_transfer` override breaks this pattern. The RBAC-generated `StdAcceptDefaultAdminTransfer` accounts struct contains only RBAC-related fields (authority, state, role PDAs) and has no knowledge of endpoint accounts. The OApp-injected handler passes `ctx.remaining_accounts`; an unconstrained, caller-supplied slice directly to `endpoint_cpi::set_delegate()`:

RUST

```
1  ::oapp::endpoint_cpi::set_delegate(  
2      ::oapp::endpoint::ID,  
3      ctx.accounts.default_admin.key(),  
4      &ctx.remaining_accounts,          // caller-supplied, no Anchor validation  
5      &seeds,  
6      SetDelegateParams { delegate: ctx.accounts.authority.key() },  
7  )?;
```

RECOMMENDATION

Consider extending `StdAcceptDefaultAdminTransfer` (or use `oapp_extend_accounts!`) to include the endpoint accounts as named fields, consistent with how every other endpoint CPI in the framework declares its accounts.

RESOLUTION

LayerZero Team - Declined. `accept_default_admin_transfer` follows the same validated `remaining_accounts` pattern as other Endpoint CPI paths; the reported inconsistency was determined not to exist.

INFO R1-I-01

RESOLVED

EnforcedOptions space vs field order mismatch.

CATEGORY Informational

LOCATION apps/oapp-app/contracts/solana/macros/src/state_types.rs#L35

REPO Round 1 - Main Review

DESCRIPTION

In `state_types.rs`, the `EnforcedOptions` struct is declared with field order `options: Vec<u8>` then `bump: u8`. Borsh serializes in declaration order, producing the on-disk layout: `discriminator(8) + options_len(4) + options_data(N) + bump(1)`. However, the `space()` function's doc comment states `Layout: discriminator (8) + bump (1) + vec_len_prefix (4) + options_data`, placing `bump` before `options`; the opposite of the actual Borsh layout.

RECOMMENDATION

Consider correcting the doc comment to reflect the actual Borsh layout: `/// Layout: discriminator (8) + vec_len_prefix (4) + options_data + bump (1)`.

RESOLUTION

LayerZero Team - Resolved.

LOW R1-L-04

RESOLVED

Missing `#[repr(u8)]` enable role seed collision.

CATEGORY Validation LOCATION `contracts/common/utils/solana/rbac/macros/src/role_type.rs#L10-L32`

REPO Round 1 - Main Review

DESCRIPTION

The `role_type_derive_impl` function in `rbac/macros/src/role_type.rs` validates only that the enum is non-empty and all variants are unit variants. It does not validate that the enum carries `#[repr(u8)]`. The `RoleType::seed()` method in `rbac/src/traits.rs` converts the role to `u8` via `Into<u8>` and uses that single byte as the PDA seed component: `[ROLE_MEMBER_SEED, state.key(), seed_byte, member.key()]`. The `#[only_role]` constraint validates authorization purely by checking PDA existence at these seeds.

Without `#[repr(u8)]` enforcement, two collision paths exist:

1. No `#[repr]` + manual `Into<u8>` collision: A developer omits `#[repr(u8)]` and writes a manual `Into<u8>` impl that maps two distinct roles to the same byte value.
1. `#[repr(u16)]` + truncation: A developer uses `#[repr(u16)]` with a variant value like `Special = 256`, then implements `Into<u8>` with `self as u8`; truncating `256` to `0`, which collides with `DefaultAdmin`.

RECOMMENDATION

Consider adding validation in `role_type_derive_impl` to verify the enum carries `#[repr(u8)]` using `input.attrs` inspection. Emit a compile-time error if the attribute is missing.

RESOLUTION

LayerZero Team - Resolved.

LOW **R1-L-05**

RESOLVED

only_role Does Not Enforce Signer.

CATEGORY Validation LOCATION contracts/common/utils/solana/rbac/macros/src/only_role.rs#L34-L44

REPO Round 1 - Main Review

DESCRIPTION

The `#[only_role]` macro currently enforces role membership by checking for a valid RoleMember PDA derived from `(state, role, authority.key())`, but it does not enforce that `authority` is a signer. This proves that the provided `authority` public key has the role, but it does not by itself prove that the caller controls that key (i.e., that it signed the transaction). This means the authorization guarantee depends on integrators correctly declaring `authority` as `Signer<'info>` (or adding an equivalent signer constraint). If they instead use a non-signer account type, the role check can be satisfied using a user-supplied pubkey.

In that case, an attacker can pass a privileged account's public key and matching role PDA without possessing its private key, resulting in unauthorized execution of role-protected instructions in downstream programs.

While all `authority` fields in this repository's OApp module are currently typed as `Signer<'info>`, from an RBAC framework perspective `#[only_role]` creates a security gap for external integrators because it does not enforce `signer` status on its own.

RECOMMENDATION

Update `#[only_role]` to enforce signer proof by default (for example via a generated `authority.to_account_info().is_signer` constraint, or a compile-time requirement that authority is `Signer<'info>`).

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-03

RESOLVED

Misleading `set_delegate` mention in macro doc.

CATEGORY Informational

LOCATION `apps/oapp-app/contracts/solana/macros/src/lib.rs#L12`

REPO Round 1 - Main Review

DESCRIPTION

The doc comment on the `#[oapp]` proc-macro in `apps/oapp-app/contracts/solana/macros/src/lib.rs` states

RUST

```
1  /// Uses the `anchor-trait` collect → generate → assemble pipeline to emit all
2  /// standard OApp instructions. Instructions with a framework-provided default
3  /// (e.g. `set_peer`, `set_delegate`) are generated automatically; instructions
4  /// without a default (e.g. `lz_receive`) must be supplied by the implementor
5  /// via `#[oapp_instruction]`.
```

`set_delegate` is not an auto-generated default OApp instruction. The `OAppInstructionSet` declared in `apps/oapp-app/contracts/solana/macros/src/instructions/oapp/mod.rs` contains 10 entries: `SetPeer`, `GetPeer`, `InitEnforcedOptions`, `SetEnforcedOptions`, `GetEnforcedOptions`, `NextNonce`, `IsComposeMsgSender`, `LzReceiveTypesInfo`, `LzReceiveTypesV2`, `LzReceive`; and no `SetDelegate`. The `endpoint_cpi::set_delegate` wrapper is only ever invoked from the unconditional `accept_default_admin_transfer` override.

RECOMMENDATION

Consider replacing `set_delegate` in the example list with an actual auto-generated default such as `get_peer` or `init_enforced_options`. Optionally consider adding a sentence noting that endpoint delegate rotation is intentionally bound to the `accept_default_admin_transfer` flow, mirroring `OAppCoreRBACUpgradeable`.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-04

RESOLVED

lz_receive_types_info accounts undocumented.

CATEGORY Best Practices

LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/lz_receive_types_info.rs, apps/oapp-app/contracts/solana/macros/src/instructions/composer/lz_compose_types_info.rs

REPO Round 1 - Main Review

DESCRIPTION

The off-chain LayerZero Executor invokes `lz_receive_types_info` with a fixed 2-account list, in this order: `oapp_account`, then the `lz_receive_types_accounts` PDA at `[LZ_RECEIVE_TYPES_SEED, oapp_account.key()]`. This is documented on the LZ docs site https://docs.layerzero.network/v2/developers/solana/oapp/overview#how-lz_receive_types_v2-works, but is not surfaced anywhere in the framework.

The framework's spec at `apps/oapp-app/contracts/solana/macros/src/instructions/oapp/lz_receive_types_info.rs` is a required override with no compile-time check on the accounts shape, so a developer can declare a context struct of any size and the framework will compile it. Mismatch with the Executor's 2-account call is a runtime failure, discovered only after deployment.

The same exists for `lz_compose_types_info`.

RECOMMENDATION

Consider promoting `lz_receive_types_info` / `lz_compose_types_info` from a required override to a Provided default with the canonical 2-account context and handler. Devs can still override via `#[oapp_instruction]` when needed, but get the correct shape for free by default.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-05

RESOLVED

Unused error variant OAppError::InvalidPeer.

CATEGORY Informational LOCATION apps/oapp-app/contracts/solana/src/errors.rs#L13

REPO Round 1 - Main Review

DESCRIPTION

`OAppError::InvalidPeer` is declared at `apps/oapp-app/contracts/solana/src/errors.rs` but is never used anywhere in the repo. It is surfaced to clients via the IDL but cannot be emitted on-chain.

RECOMMENDATION

Consider removing the variant, or adding the intended `set_peer` validation that was meant to raise it.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-06

ACKNOWLEDGED

RBAC role-admin mapping is fixed at compile time.

CATEGORY Informational LOCATION contracts/common/utils/solana/rbac/macros/src/instructions/grant_role.rs#L32

REPO Round 1 - Main Review

DESCRIPTION

The Solana RBAC supports hierarchical role admins via `RoleType::role_admin()` (`contracts/common/utils/solana/rbac/src/traits.rs`), and `grant_role` / `revoke_role` use it when deriving the admin PDA

RUST

```
1 // contracts/common/utils/solana/rbac/macros/src/instructions/grant_role.rs
2 let role_admin_seed = quote!(&::rbac::traits::RoleType::seed(&role_type::role_admin(&ms.role)));
```

But `role_admin()` is resolved at macro-expansion time; the role→admin mapping is fixed in source. There is no on-chain instruction analogous to OpenZeppelin's `_setRoleAdmin(role, adminRole)`, which EVM contracts (including `AccessControlDefaultAdminRulesUpgradeable`, the base of `0AppCoreRBACUpgradeable`) can expose to change role graphs at runtime. Changing a role's admin on Solana requires a full program upgrade. Additionally, `role_admin(&self)` → Self receives no context/accounts, so even overriding the trait can't read on-chain state to implement runtime mutability.

RECOMMENDATION

Either consider adding an on-chain `set_role_admin` instruction (full EVM parity) or documenting the divergence explicitly; role hierarchy is static and changing it requires a program upgrade.

RESOLUTION

LayerZero Team - Acknowledged. The compile-time role-admin mapping was intentionally retained.

LOW R1-L-06

RESOLVED

Fully-qualified attribute paths not matched.

CATEGORY Validation

LOCATION contracts/common/framework/solana/anchor-trait/src/assembly.rs#L78, contracts/common/framework/solana/anchor-trait/src/overrides.rs#L179

REPO Round 1 - Main Review

DESCRIPTION

Two path predicates in `anchor-trait` use `syn::Path::is_ident`, which is strict-1-segment. Qualified forms of the same attribute slip through.

1. `#[program]` dedup (`assembly.rs`)

RUST

```
1 let mod_attrs: Vec<_> =  
2   mod_attrs.iter().filter(|a| !a.path().is_ident("program")).collect();
```

`#[anchor_lang::prelude::program]` or `#[::anchor_lang::prelude::program]` is not filtered, so the generated module carries two `#[program]` attributes and Anchor's program macro runs twice.

1. `#[<domain>_instruction]` override recognition (`overrides.rs`)

RUST

```
1 /// Returns `true` if `attr`'s last path segment matches the configured attribute name.  
2 fn is_override_attr(attr: &Attribute, cfg: OverrideConfig) → bool {  
3     attr.path().is_ident(cfg.attr_name)  
4 }
```

Doc says last-segment; code uses `is_ident`. `#[crate::oapp_instruction]` bypasses recognition - if the handler name doesn't collide with a managed instruction, it passes through as a regular item and the framework silently emits the default.

RECOMMENDATION

Consider an exact-match allowlist of Anchor's canonical paths for `#[program]`

RUST

```
1 #[program]  
2 #[anchor_lang::program]  
3 #[::anchor_lang::program]  
4 #[anchor_lang::prelude::program]  
5 #[::anchor_lang::prelude::program]
```

For the override marker recognition in `is_override_attr`, compare only the final path segment:

RUST

```
1 // overrides.rs  
2 attr.path().segments.last().is_some_and(|seg| seg.ident == cfg.attr_name)
```

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-07

RESOLVED

Marker attribute macro bodies are unreachable.

CATEGORY Informational

LOCATION <https://app.notion.com/p/Audit-Repository-52edb1aef30f4b8cb3c8df705e55f232?pvs=21>

REPO Round 1 - Main Review

DESCRIPTION

The framework declares three `#[proc_macro_attribute]` stubs that users apply to override handlers: `oapp_instruction()` at `apps/oapp-app/contracts/solana/macros/src/lib.rs#L29`, `composer_instruction()` at `apps/oapp-app/contracts/solana/macros/src/lib.rs#L56`, and `rbac_instruction()` at `contracts/common/utis/solana/rbac/macros/src/lib.rs#L56`. Each body today is a trivial identity pass-through (`item`).

Despite being declared as proc-macro attributes, these functions never execute in normal usage. Attribute expansion is outer-first: the enclosing `#[oapp::oapp]`, `#[oapp::composer]`, or `#[::rbac::rbac]` attribute consumes the whole module as a `TokenStream` and strips inner `#[oapp_instruction]` / `#[composer_instruction]` / `#[rbac_instruction]` markers via `Instruction0verrides::wrap_handler()` before Rust's attribute resolver sees them. The markers are recognised purely by string match on `OverrideConfig::attr_name` in `anchor-trait`'s `overrides.rs` - the stubs themselves are never invoked as macros.

This is safe today because each body is identity. However, if a future maintainer adds logic to any of these bodies (e.g. validation, rewriting, diagnostics), that logic will silently not run in the normal flow. Tests applied via the enclosing domain macro will not exercise the stub.

RECOMMENDATION

Add a comment to each stub documenting that the body is unreachable under normal usage, and that logic intended to affect real expansions must be placed in the enclosing domain macro (`oapp::expand()`, `composer::expand()`, `rbac::expand()`) or in the shared `Instruction0verrides` pipeline - not in the stub.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-08

ACKNOWLEDGED

Duplicate use super::* in generated module.

CATEGORY Informational LOCATION contracts/common/framework/solana/anchor-trait/src/assembly.rs#L90

REPO Round 1 - Main Review

DESCRIPTION

`Assembly::assemble()` always injects `use super::*;` as the first item inside the generated program module. When two domain macros are stacked - e.g. `#[oapp::oapp]` followed by `#[::rbac::rbac]` - each macro runs its own `assemble()` pass. The second pass re-emits the module with a fresh `use super::*;` while preserving the prior pass's `use super::*;` as part of `filtered_items()`. As a result, the final expanded module contains two identical `use super::*;` imports.

Concretely, after `#[oapp::oapp]` expands in `apps/oapp-app/contracts/solana/macros/src/oapp.rs`, its output contains `#[::rbac::rbac(...)] mod counter { use super::*; ... }`. When the RBAC attribute is then expanded and its `Assembly::assemble()` re-wraps the module, the preserved body (passed through `Instruction0overrides::filtered_items()`) still carries the original `use super::*;`, and a second one is prepended by the new emission. The duplication is harmless to compilation because `use` declarations are idempotent, but it is unnecessary noise in the expanded output (visible via `cargo expand`) and grows linearly with the number of stacked domain macros (e.g. `#[oapp]` + `#[composer]` + `#[rbac]` would produce three copies).

RECOMMENDATION

Either fix or acknowledge.

RESOLUTION

LayerZero Team - Acknowledged. Duplicate imports were accepted as harmless.

MEDIUM R1-M-01

RESOLVED

OApp macro rejects module inner attributes.

CATEGORY Logical Error LOCATION contracts/common/framework/solana/anchor-trait/src/assembly.rs#L86 REPO Round 1 - Main Review

DESCRIPTION

The `anchor-trait` `Assembly::assemble()` in `contracts/common/framework/solana/anchor-trait/src/assembly.rs` re-emits the user-authored module's attributes (`ItemMod.attrs`) without distinguishing outer style (`#![...]`) from inner style (`#![...]`). Both are dumped uniformly via `#![mod_attrs]*` at a position outside the generated program module

RUST

```
1  quote! {
2      #prelude
3      #wrapper_defs
4      #default_impls
5      #extend_macro
6
7      #(#mod_attrs)*           // ← inner attrs (#![...]) land here, outside the mod
8      #extra_mod_attrs
9      #[program]
10     #mod_vis mod #mod_ident {
11         use super::*;
12         #filtered_items
13         #entrypoints
14     }
15 }
```

`quote!` preserves `AttrStyle` on round-trip, so a developer-written `#![X]` is re-emitted as `#![X]` in the new position. Because `#prelude`, `#wrapper_defs`, `#default_impls`, and `#extend_macro` always emit tokens before this point, the inner attribute is never in a legal position (first tokens of an enclosing item). `rustc` rejects it with a hard compile error:

TEXT

```
1  error: an inner attribute is not permitted in this context
2      → src/lib.rs
3      |
4  175 | #[oapp::oapp(state = CounterState, role_type = CounterRoleType)]
5      | ----- the inner attribute doesn't annotate this module
6  176 | pub mod counter {
7  177 |     #![deny(clippy::arithmetic_side_effects)]
8      |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
9      |
10     = note: inner attributes, like `#![no_std]`, annotate the item enclosing them
```

This blocks developers from using idiomatic module-scoped Rust smart-contract hardening pragmas inside the `#[oapp]` module.

RECOMMENDATION

Consider partitioning `mod_attrs` by `syn::AttrStyle` and route each style to the position where `rustc` accepts it: outer attrs stay outside the generated module (they annotate the module from outside); inner attrs move to the first tokens inside the generated module body (they annotate the module from inside). Both land on `mod #mod_ident` semantically - the `!` is a positional marker, not a functional distinction.

RUST

```
1  // contracts/common/framework/solana/anchor-trait/src/assembly.rs
2  let (inner_attrs, outer_attrs): (Vec<_>, Vec<_>) = mod_attrs
3      .into_iter()
4      .partition(|a| matches!(a.style, syn::AttrStyle::Inner(_)));
5
6  quote! {
7      #prelude
8      #wrapper_defs
9      #default_impls
10     #extend_macro
11
12     #(#outer_attrs)*
13     #extra_mod_attrs
14     #[program]
15     #mod_vis mod #mod_ident {
16         #(#inner_attrs)*
17         use super::*;
18         #filtered_items
19         #entrypoints
20     }
21 }
```

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-09

RESOLVED

endpoint_cpi Helpers Can Panic on Short Slices.

CATEGORY Validation LOCATION apps/oapp-app/contracts/solana/src/endpoint_cpi.rs#L40

REPO Round 1 - Main Review

DESCRIPTION

Several CPI helpers in `oapp::endpoint_cpi` index into the caller-supplied accounts slice before verifying its length. As a result, undersupplied `remaining_accounts` may trigger uncontrolled aborts (e.g., `ProgramFailedToComplete`) instead of returning a structured error such as `AccountNotEnoughKeys`.

While malformed input still results in transaction failure, this unclean failure mode degrades debuggability and integration ergonomics. The pattern appears to be consistent across multiple helpers, including `register_oapp`, `set_delegate`, `send`, `clear`, `send_compose`, and `clear_compose`.

RECOMMENDATION

Consider adding explicit slice-length checks before any positional `accounts[...]` access in all `endpoint_cpi` helpers, and return a standard error on undersupplied input.

RESOLUTION

LayerZero Team - Resolved.

LOW R1-L-07

RESOLVED

Override Handler Cannot Access Nested Module Helpers.

CATEGORY Logical Error LOCATION contracts/common/framework/solana/anchor-trait/src/overrides.rs#L119-L133 REPO Round 1 - Main Review

DESCRIPTION

`anchor-trait` relocates override handlers into generated sibling wrapper modules, which changes their lexical scope relative to the user-authored `#[oapp]` / `#[rbac]` module. As a result, an overridden instruction such as `lz_receive` cannot reliably access a nested helper module defined inside the same macro-managed module.

For example:

```
TEXT
1  #[oapp::oapp(...)]
2  pub mod my_oapp {
3      mod utils {
4          pub fn helper() { ... }
5      }
6
7      #[oapp_instruction]
8      pub fn lz_receive(...) → Result<()> {
9          utils::helper();
10         Ok(())
11     }
12 }
```

The code above cannot compile because `utils` is inside `my_oapp`, but the relocated override handler will be placed in a sibling module rather than remaining inside `my_oapp`, roughly like below

```
TEXT
1  mod __oapp_instruction__lz_receive_handler {
2      use super::*;
3      pub fn lz_receive(...) → Result<()> {
4          utils::helper(); // now looked up from this sibling wrapper scope
5          Ok(())
6      }
7  }
8
9  #[program]
10 pub mod my_oapp {
11     use super::*;
12
13     mod utils {
14         pub fn helper() { ... }
15     }
16
17     pub fn lz_receive(...) → Result<()> {
18         __oapp_instruction__lz_receive_handler::lz_receive(...)
19     }
20 }
```

For a workaround, developers who need nested helper modules for overridden handlers must either inline the logic inside the handler itself or move the helper module outside the `#[oapp]` module into outer file scope or another utility module that remains visible to the generated wrapper modules.

RECOMMENDATION

One option is to avoid moving override handlers into isolated wrapper modules, which would preserve the user module's lexical scope and allow overridden handlers to access nested same-module helper modules naturally.

Alternatively, document this behavior clearly and warn developers that helper modules used by overridden handlers should be placed outside the macro-managed module, such as in outer scope or a separate utility module.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-10

RESOLVED

Events omit the state instance pubkey.

CATEGORY Events LOCATION contracts/common/utils/solana/rbac/src/events.rs#L5-L28

REPO Round 1 - Main Review

DESCRIPTION

The rbac framework documents multi-instance programs as a supported feature - the `RoleMember` PDA is seeded on `state.key()` so different state instances get isolated role namespaces. However, none of the emitted events include the `state.key()` (nor any other instance identifier), so off-chain consumers cannot tell which instance an event refers to.

The affected event types are:

- `RoleGranted { role, account, sender }` - emitted by `grant_role()`
- `RoleRevoked { role, account, sender }` - emitted by `revoke_role()`, `renounce_role()`, and `accept_default_admin_transfer()` (for the outgoing admin)
- `DefaultAdminTransferStarted { new_admin }` - emitted by `begin_default_admin_transfer()`

In a multi-instance program, two independent `grant_role()` calls on two different state instances produce identical `RoleGranted` payloads if they happen to grant the same role to the same account. An off-chain indexer, governance dashboard, or alerting system subscribing to these events has no reliable way to attribute each event to the instance it originated from. The same concern applies symmetrically to the OApp-side events `PeerSet` and `EnforcedOptionsSet`, which omit the `oapp.key()` they mutated.

RECOMMENDATION

Add a `state: Pubkey` field to each `#[event]` struct in the rbac crate and the oapp crate, and populate it from `ctx.accounts.default_admin.key()` (or the equivalent state field) at every `emit_cpi!` site.

RESOLUTION

LayerZero Team - Resolved.

INFO **R1-I-11**

RESOLVED

The default_admin_role should not be dynamic.

CATEGORY Warning LOCATION contracts/common/utils/solana/rbac/src/traits.rs#L60-L62

REPO Round 1 - Main Review

DESCRIPTION

The `rbac` code intentionally forbids granting or revoking/renouncing the default admin role. But if `RoleType::default_admin_role` returns a different value because of override or upgrade, this invariant can be bypassed. For example, the new `default_admin_role` can be set as an already existing role. This will silently promote all the members of the previous role.

The `RoleMember` for the previous default admin will become unreachable. This will also create the following asymmetry:

- `begin_default_admin_transfer` - remains callable only by the real default_admin
- `accept/grant/revoke/renounce` - can be called by any of the holders of the new role

RECOMMENDATION

Consider documenting that `default_admin_role` must not change.

RESOLUTION

LayerZero Team - Resolved.

LOW **R1-L-08**

ACKNOWLEDGED

Peer/options accounts cannot be closed to reclaim rent.

CATEGORY Best Practices

LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/set_peer.rs#L44-L50

REPO Round 1 - Main Review

DESCRIPTION

The `set_peer()` instruction generated by the `#[oapp]` macro creates an `OAppPeer` PDA per `(oapp, eid)` pair via `init_if_needed`, paying rent from the `payer` signer. The generated `StdSetPeer` accounts struct has no `close = ...` directive, and no separate `close_peer()` / `remove_peer()` / `disable_peer()` instruction is emitted by the macro. The `OAppPeer` struct itself contains only `address: [u8; 32]` and `bump: u8` - there is no enabled/disabled flag.

As a result, once an OApp admin enables a remote chain by calling `set_peer()`, the only path to "disable" that peer on Solana is to call `set_peer()` again with `peer = [0u8; 32]`. The consequences are:

1. The rent-paid PDA stays allocated forever; the SOL paid at initialization is unrecoverable.

1. The emitted `PeerSet { eid, peer: [0; 32] }` event is indistinguishable from "freshly initialized but not yet configured" by off-chain indexers.

This is the framework-level macro used by every OApp built on top of it, so the issue compounds across chains/EIDs and across deployments.

This is also true for the `EnforcedOptions` accounts

RECOMMENDATION

Add a `close_peer()` (or `remove_peer()`) instruction to the `#[oapp]` macro's default impls that

- Requires `default_admin_role` (same RBAC as `set_peer()`).
- Uses Anchor's `close = <rent_destination>` constraint on the `peer: Account<'info, OAppPeer>` so that the lamports are returned to the admin / payer.
- Emits a distinct event (e.g. `PeerClosed { eid }`) so off-chain indexers can differentiate "explicitly disabled" from "set to zero address" from "never initialized".

Alternatively, change `SetPeerParams::peer` to `Option<[u8; 32]>` and close the PDA when `None` is supplied. Apply this change to `EnforcedOptions` as well

RESOLUTION

LayerZero Team - Acknowledged. The current interface was intentionally retained and no close path was added.

INFO R1-I-12

RESOLVED

RoleRevoked.sender doc copy-pasted from grant.

CATEGORY Informational LOCATION contracts/common/utils/solana/rbac/src/events.rs#L19

REPO Round 1 - Main Review

DESCRIPTION

The `sender` field on the `RoleRevoked` event carries a doc comment that describes the field as if it belonged to `RoleGranted`. The comment refers to the operator who initiated a grant, even though the event is emitted on revocation.

RUST

```
1  #[event]
2  pub struct RoleGranted {
3      pub role: u8,
4      pub account: Pubkey,
5      /// The operator who initiated the grant (named `sender` for EVM parity).
6      pub sender: Pubkey,
7  }
8
9  #[event]
10 pub struct RoleRevoked {
11     pub role: u8,
12     pub account: Pubkey,
13     /// The operator who initiated the grant (named `sender` for EVM parity).
14     pub sender: Pubkey,
15 }
```

RECOMMENDATION

Consider updating the doc comment on `RoleRevoked.sender` to reflect the revoke semantic, for example

RUST

```
1  /// The operator who initiated the revoke (named `sender` for EVM parity).
2  pub sender: Pubkey,
```

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-13

RESOLVED

Clarify endpoint program check in CPI wrappers.

CATEGORY Documentation

LOCATION Solana%20App%20Findings%20Table/I-13%2034f8bda5828c81738042d92907b638f8.html

REPO Round 1 - Main Review

DESCRIPTION

Each CPI wrapper in `endpoint_cpi.rs` (`register_oapp()` , `set_delegate()` , `send()` , `quote()` , `clear()` , `send_compose()` , `clear_compose()`) carries a doc comment of the form

`accounts[0]` must be the Endpoint program (validated by `construct_context`).

The wording could be read to imply that `construct_context` verifies `accounts[0]` against the canonical LayerZero Endpoint program. In practice, the check inside `construct_context` (generated by `cpi_helper` 's `CpiContext` derive - see `LayerZero-v2/packages/layerzero-v2/solana/programs/libs/cpi-helper/src/lib.rs#L51-L53`) is:

RUST

```
1  if (program_id != accounts[0].key()) {
2      return Err(InvalidProgramId.into());
3  }
```

`program_id` is bound to the `endpoint_program: Pubkey` parameter that the caller supplied. So the check confirms that the two caller-supplied values are consistent with each other, but it does not by itself establish that either one is the real Endpoint. The actual identity guarantee comes from wherever the outer OApp instruction sources `endpoint_program` - typically a hardcoded constant or a stored, RBAC-gated config field.

This is not a vulnerability in the wrappers themselves; the design simply pushes the trust boundary one layer up to the caller. The current docstring may give readers (and future maintainers building OApps on top of these wrappers) a slightly stronger impression of where the guarantee lives than is accurate.

RECOMMENDATION

Consider rewording the docstrings so the trust boundary is explicit. For example

TEXT

```
1  /// # Accounts validation
2  /// - `accounts[0]` must equal the caller-supplied `endpoint_program` parameter.
```

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-14

RESOLVED

Misleading Code Comment.

CATEGORY Documentation LOCATION contracts/common/framework/solana/anchor-trait/src/lib.rs#L16

REPO Round 1 - Main Review

DESCRIPTION

The comment in `anchor-trait/src/lib.rs` states, "See `spec.md` for the full specification." However, the repository does not include a `spec.md` file, making the comment misleading.

RECOMMENDATION

Update the comment to reflect the current repository structure, or add a `spec.md` file.

RESOLUTION

LayerZero Team - Resolved.

Endpoint CPI wrappers rely on account ordering.

CATEGORY Suggestion

LOCATION apps/oapp-app/contracts/solana/src/endpoint_cpi.rs#L24-L187

REPO Round 1 - Main Review

DESCRIPTION

The CPI wrappers in `endpoint_cpi.rs` (`register_oapp()` , `set_delegate()` , `send()` , `quote()` , `clear()` , `send_compose()` , `clear_compose()`) all rely on the caller passing the `accounts` slice in a specific positional order

- `accounts[0]` must be the endpoint program.
- `accounts[1..N]` must match the field declaration order of the corresponding `cpi::accounts::*` struct in `endpoint-interface` .

This positional dependency comes from the `cpi_helper::CpiContext` derive macro at `LayerZero-v2/packages/layerzero-v2/solana/programs/lib/cpi-helper/src/lib.rs#L22-L33` , which generates a `construct_context()` implementation that pops accounts from the slice in field-declaration order and assigns each to the matching CPI accounts field by position.

The wrappers in `endpoint_cpi.rs` further encode this dependency via hardcoded indices, e.g.:

- `register_oapp()` checks `accounts[2] == oapp` because `oapp` is the second field of `RegisterOApp` (after `payer`).
- `set_delegate()` , `send()` , `clear()` , `send_compose()` , and `clear_compose()` each check `accounts[1]` because their respective signer/identity field is declared first in the struct.

If an OApp developer building on top of this framework is unaware of the strict positional contract, they could easily pass the `accounts` slice in a wrong order. The wrapper's own check would only catch the mismatch at the single hardcoded index it inspects (e.g., `accounts[2]` for `register_oapp()`); other slots could silently land in the wrong field. The endpoint's downstream account validation would eventually reject the malformed CPI, but the resulting error (a seed-derivation or ownership failure) is far less obvious than a layout mismatch and complicates debugging.

The `endpoint-interface` git dependency in `Cargo.toml` is pinned by SHA, so the upstream struct order cannot drift unexpectedly. The remaining risk is purely a developer-awareness one: the wrappers' security guarantees depend on a layout contract that is implicit and undocumented at the call site.

RECOMMENDATION

Consider adding a brief comment on each wrapper (or at the top of the module) documenting the positional dependency on the corresponding `cpi::accounts::*` struct. For example

TEXT

```
1  /// NOTE: `accounts[2]` corresponds to the `oapp` field of `RegisterOApp` in
2  /// `endpoint-interface`. The full slice must follow the struct's field order:
3  /// [endpoint_program, payer, oapp, oapp_registry, system_program, ...remaining].
```

This makes the implicit assumption explicit at the call site, helping OApp developers lay out their accounts correctly the first time.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-16

ACKNOWLEDGED

OApp initialize() is frontrun-hijackable.

CATEGORY Frontrunning LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/mod.rs#L27

REPO Round 1 - Main Review

DESCRIPTION

The `#[oapp]` macro framework generates a set of admin-gated instructions (`set_peer()` , `init_enforced_options()` , `set_enforced_options()` , `next_nonce()` , etc.), but does not generate or scaffold an `initialize()` instruction. Each OApp implementer must hand-roll their own initialization handler that creates the OApp `state` PDA, calls `endpoint::register_oapp()` via CPI, and seeds the initial RBAC default-admin role.

Solana's runtime delays the visibility of newly-deployed programs by `DELAY_VISIBILITY_SLOT_OFFSET = 1` slot, so every OApp deployment exposes an unavoidable window between `solana program deploy` confirmation and the deployer's own `initialize()` tx landing in a subsequent block. Atomic deploy-plus-initialize is not achievable through any client-side mechanism (including Jito bundles, which execute within a single slot and therefore cannot invoke a program deployed in that same slot). During the window, any signer can submit a competing `initialize()` and:

1. Become the OApp's `DefaultAdmin` (typically via `#[rbac::init_default_admin(... admin = payer.key())]` , which the framework's RBAC layer publishes as the recommended pattern).
1. Set themselves (or any address) as the LayerZero endpoint `delegate` through the freely-chosen `register_oapp` parameter.
1. Permanently lock out the legitimate deployer - the `state` account's `init` constraint succeeds exactly once.

After the race resolves in the attacker's favor, they control all admin-gated configuration: `set_peer()` , `set_send_library()` , `set_receive_library()` , `init_enforced_options()` , `grant_role()` , `begin_default_admin_transfer()` , etc.

The framework provides no built-in mechanism to mitigate this. There is no macro-generated initializer, no attribute that binds initialization to the program's upgrade authority, and no mention of the hazard in the framework documentation. Protocol teams integrating the `#[oapp]` macro must independently know about Solana's deploy-and-init race and harden their handler accordingly - a non-obvious requirement for teams whose primary background is EVM, where proxy constructors atomize the same operation.

RECOMMENDATION

Either (a) document the hazard in the OApp framework guide and demonstrate the upgrade-authority constraint pattern, or (b) provide a framework-level attribute (e.g. `#[oapp::deployer_only_init]`) that binds `initialize()` to the program's upgrade authority by adding a `program_data: Account<'info, ProgramData>` to the accounts struct with a `program_data.upgrade_authority_address = Some(deployer.key())` constraint. The constraint is robust regardless of race-window size because every non-deployer `initialize()` reverts.

RESOLUTION

LayerZero Team - Acknowledged. The framework-level initialization risk remains; Console OFT handles it at the application level.

RBAC docs reference non-existent #[grant_role].

CATEGORY Documentation

LOCATION contracts/common/utils/solana/rbac/src/lib.rs#L42

REPO Round 1 - Main Review

DESCRIPTION

The crate-level documentation in `lib.rs` instructs integrators to use macros that do not exist in the current API. Both the prose ("Quick Start") and the runnable-looking example show a per-instruction attribute macro `#[grant_role(...)]` (and references `#[revoke_role]` , `#[renounce_role]`) that is not exported by `rbac_macros` .

RUST

```
1  //! ## Quick Start
2  //!
3  //! 1. Define your role enum with `#[derive(RoleType)]`
4  //! 2. Define your `RoleMember` account with `bump`, `role`, and `account` fields
5  //! 3. Use `#[grant_role]`, `#[revoke_role]`, `#[renounce_role]` macros on your instruction structs
6  //!
7  //! ## Example
8  //!
9  //! ``ignore
10 //! ...
11 //! // The macro auto-generates the state field and all RBAC accounts
12 //! #[grant_role(program_state = Store, role_member = RoleMember, role_type = RoleType)]
13 //! pub struct GrantRole<'info> {}
14 //! ``
15
16
17 The actual API exported from `rbac_macros` (per `macros/src/lib.rs` and the README) is:
18
19 - `#[rbac(state = ..., role_type = ...)]` - applied to the `#[program]` *module*, generates all six RBAC instructions
20 - `#[only_role(...)]`, `#[init_default_admin(...)]` - accounts-struct attribute macros
21 - `#[derive(RoleType)]`, `#[derive(DefaultAdmin)]`
22 - `#[rbac_instruction]` - override marker
23
24 There is no per-instruction `#[grant_role]`, `#[revoke_role]`, or `#[renounce_role]` attribute. The
25 `grant_role`/`revoke_role`/`renounce_role` instructions are generated by the module-level `#[rbac]` macro, not by per-struct
26 attributes.
27
28 **Recommendation**
29 Consider rewriting the Quick Start and Example sections in `lib.rs` to reflect the real API.
```

RECOMMENDATION

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-18

RESOLVED

deserialize_alt Can Panic Via unwrap.

CATEGORY Informational

LOCATION apps/oapp-app/contracts/solana/src/common.rs#L135

REPO Round 1 - Main Review

DESCRIPTION

`deserialize_alt` contains a panic-style code path when reading ALT account data. In `common.rs`, it invokes `alt.try_borrow_data().unwrap()` prior to deserializing the address lookup table; as a result, a borrow failure causes an uncontrolled abort rather than returning a structured program error.

As a result, transaction failure occurs via a panic rather than returning `InvalidAddressLookupTable`, leading to inconsistent error-handling behavior.

RECOMMENDATION

Consider replacing the `unwrap` with proper error propagation, and returning a controlled error on borrow failure.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-19

RESOLVED

Hardcoded crate path in declare_instructions.

CATEGORY Compatibility

LOCATION contracts/common/framework/solana/anchor-trait/src/lib.rs#L98, #L121

REPO Round 1 - Main Review

DESCRIPTION

`declare_instructions!()` expands generated code with explicit `::anchor_trait` paths for both `InstructionSet` and `InstructionSpec`. This couples downstream users to one exact dependency name in `Cargo.toml`. If an integrating macro crate renames the dependency key, such as `anchor-trait` to `my-anchor-trait`, local imports can be updated successfully but the macro expansion still fails because `::anchor_trait` is no longer resolvable.

RECOMMENDATION

Replace hardcoded `::anchor_trait` references in `declare_instructions!()` with `\$crate`-qualified paths such as `$crate::InstructionSet` and `$crate::InstructionSpec`. This preserves correct resolution to the defining crate while remaining compatible with renamed dependencies in downstream crates.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-20

RESOLVED

Arbitrary marker accepted instead of (ctx).

CATEGORY Compatibility

LOCATION contracts/common/framework/solana/anchor-trait/src/lib.rs#L60, #L85

REPO Round 1 - Main Review

DESCRIPTION

`declare_instructions!()` documents `(ctx)` as the syntax that selects the `spec(&ctx)` dispatch path, but the matcher accepts any identifier in that position. As a result, inputs such as `(banana)` are treated the same as `(ctx)` and silently route to the ctx-passing branch. This makes the macro interface looser than its documented contract and can hide typos or misunderstandings in downstream macro crates.

RECOMMENDATION

Restrict the ctx-passing branch to the literal `(ctx)` marker and reject any other identifier with an explicit compile-time error. This keeps the accepted syntax aligned with the documented API and prevents accidental acceptance of mistyped markers.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-21

RESOLVED

Instruction aliases emit duplicate entrypoints.

CATEGORY Documentation

LOCATION contracts/common/framework/solana/anchor-trait/src/generation.rs#L188-L198

REPO Round 1 - Main Review

DESCRIPTION

The framework derives each generated entrypoint name from the instruction's canonical name. If two instruction variants are intentionally aliased to the same module in `declare_instructions!()`, they share the same canonical name and therefore generate the same public entrypoint function. This makes instruction aliases unsupported in practice, since code generation emits colliding entrypoints for both variants. In modules with default implementations, the same `spec()` and generator will also usually emit the same default accounts struct and handler impl twice. Even if an implementer forced different struct names through custom naming tricks, the generated entrypoints would still collide because the entrypoint name is tied to the shared canonical instruction name.

RECOMMENDATION

Clearly document that each instruction variant must use a unique module identity.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-22

RESOLVED

Docs understate default impl remains available.

CATEGORY Documentation

LOCATION contracts/common/framework/solana/anchor-trait/README.md#L36, #L207, #L220

REPO Round 1 - Main Review

DESCRIPTION

The documentation says the default accounts struct is always emitted and describes account extension as reusing the base implementation, but it does not state clearly enough that the original default implementation remains available after extension as well. An integrator can reasonably read the extension flow as a replacement path and assume the default implementation is no longer reachable once an extended accounts type is introduced. In practice, the framework emits both the original default struct and handler impl and the extended variant, so extension is additive rather than exclusive.

RECOMMENDATION

Update the documentation to state explicitly that extending default accounts does not replace or disable the original default implementation. The `Provided` and `Extending Default Accounts` sections should emphasize that both the original default struct and its handler impl remain emitted and usable unless the integrator deliberately routes all usage through the extended type.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-23

RESOLVED

Default accounts generics are not supported.

CATEGORY Compatibility

LOCATION contracts/common/framework/solana/anchor-trait/src/generation.rs#L84, #L177

REPO Round 1 - Main Review

DESCRIPTION

Generic parameters on default accounts types are not supported by the framework's code generation paths. In the extend-accounts path, the generated macro reconstructs the type as `pub struct $name<'info>` and `impl $name<'_>`, which drops any additional generic parameters from the original default definition. In the default entrypoint path, the generated context and dispatch use only the bare accounts ident via `Context<#accounts>` and `#accounts::#handler_fn`, so a default accounts type such as `StdShared<'info, T>` cannot be instantiated correctly. As a result, default account definitions with extra generics may compile as standalone definitions but fail once the framework generates the corresponding entrypoint or extended variant.

RECOMMENDATION

Choose one explicit behavior and document it. If generic default account types are intended to be supported, preserve full generic type information in both the default entrypoint and extend-accounts generation paths rather than reducing the type to a bare ident or hardcoded `<'info>`. If they are not intended to be supported, validate and reject extra generics beyond the expected `'info` form with a clear compile-time error and document that restriction.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-24

RESOLVED

Field-Injection Macros Always Append New Fields.

CATEGORY Documentation

LOCATION contracts/common/utils/solana/rbac/README.md#L103

REPO Round 1 - Main Review

DESCRIPTION

The README states that `#[only_role]` appends `role_member` only if that field does not already exist, but the implementation does not perform that check. In `contracts/common/utils/solana/rbac/README.md#L103`, the generated field is described as being appended conditionally; however, the macro in `only_role.rs` unconditionally does `fields.named.push(new_field)`. The same pattern exists in `init_default_admin.rs` for `admin_role_member`.

If a developer manually defines one of these fields and also applies the macro, the expansion produces duplicate field names, causing the struct to fail compilation. Although this does not pose a runtime risk, it is inconsistent with the README and may lead to confusing compile-time errors

RECOMMENDATION

Update the README to document this behavior.

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-25

RESOLVED

OAppInfo can be silently uninitialized.

CATEGORY Compatibility

LOCATION apps/oapp-app/contracts/solana/docs/oapp-info.md#L283

REPO Round 1 - Main Review

DESCRIPTION

`OAppInfo` is part of the shared OApp SDK integration contract, but its initialization is only an implicit expectation placed on application developers. The framework documents that developers should create the `OAppInfo` PDA and call `init_oapp_info!()` inside their own `initialize` instruction, but `#[oapp]` does not enforce this and does not generate a default initialization path. As a result, an OApp can be deployed in a functionally working on-chain state while still lacking the metadata required by the standardized SDK. If a developer is unaware of this requirement and later revokes upgrade authority, the deployment can become permanently incompatible with the shared SDK with no straightforward remediation path.

RECOMMENDATION

Consider adding an `initialize` macro which requires `OAppInfo` PDA to exist by default and provide an explicit opt-out for programs that intentionally do not want SDK compatibility. This approach protects devs from unknowingly creating their OApp without `OAppInfo` PDA, while still keeping maximum flexibility

RESOLUTION

LayerZero Team - Resolved.

INFO R1-I-26

RESOLVED

Generated Code Is Susceptible To Name Shadowing.

CATEGORY Compatibility

LOCATION contracts/common/framework/solana/anchor-trait/src/generation.rs#L190,
contracts/common/framework/solana/anchor-trait/src/generation.rs#L195

REPO Round 1 - Main Review

DESCRIPTION

The code generated by `anchor-trait` (including RBAC/OApp macros) relies on unqualified symbols such as `Result`, `Context`, and other Anchor types/macros resolved from the caller's scope. While RBAC documentation notes that `anchor_lang::prelude::*` is expected to be in scope, this does not prevent standard Rust name shadowing from affecting macro expansion.

In particular, generated entrypoints use bare `Result<T>` rather than a fully qualified path (e.g., `anchor_lang::Result<T>`). Additionally, the assembled program module imports all parent-scope items via use `super::*`, allowing any user-defined aliases (e.g., `type Result<T> = std::result::Result<T, ()>;`) to propagate into the generated module.

As a result, if a developer defines a conflicting alias such as `Result`, the generated code may resolve to the shadowed type instead of the intended Anchor type. This can lead to compile-time failures, even when user-authored handlers explicitly return `anchor_lang::Result`. The issue therefore stems from a macro hygiene limitation, where generated code depends on caller-scope name resolution rather than fully qualified paths.

RECOMMENDATION

Use fully qualified paths (e.g., `anchor_lang::Result`, `anchor_lang::Context`) in generated code to avoid reliance on caller-scope resolution and reduce the risk of name shadowing. Alternatively, explicitly document that shadowing core types (such as `Result`) in the parent scope is unsupported when using these macros.

RESOLUTION

LayerZero Team - Resolved.

INFO R2-I-01

RESOLVED

Unconstrained sender in setup_default_admin!.

CATEGORY Events LOCATION contracts/common/utils/solana/rbac/macros/src/setup_default_admin.rs#L102

REPO Round 2 - Main Review

DESCRIPTION

`setup_default_admin!(ctx, state, admin, role_type, sender)` parses `sender` as an arbitrary `Expr` and forwards it unmodified into the `RoleGranted` event

RUST

```
1 emit_cpi!(::rbac::events::RoleGranted {
2     state: #ctx.accounts.#state.key(),
3     role: <#role_type as ::rbac::traits::RoleType>::default_admin_role().into(),
4     account: admin_pubkey,
5     sender: #sender,           // arbitrary developer expression
6 });
```

Nothing constrains `sender` to a `Signer`-typed field, the payer, or any account related to the grant. A caller can pass `Pubkey::default()`, `system_program.key()`, or any pubkey, and the resulting event is well-formed and indexable. This is the only `RoleGranted` emission site where `sender` is not hard-coded to `ctx.accounts.authority.key()`, breaking the convention used by `grant_role` and `accept_default_admin_transfer`.

RECOMMENDATION

Consider enforcing that the passed sender is a signer.

RESOLUTION

LayerZero Team - Resolved.

INFO R2-I-02

RESOLVED

clear wrapper docs imply OApp-only signer.

CATEGORY Validation LOCATION apps/oapp-app/contracts/solana/src/endpoint_cpi.rs#L283 REPO Round 2 - Main Review

DESCRIPTION

The Endpoint program's `Clear` accounts struct authorizes the `signer` account through a disjunction

```
RUST
1 constraint = signer.key() == params.receiver
2 || signer.key() == oapp_registry.delegate @LayerZeroError::Unauthorized
```

The endpoint deliberately allows either the OApp PDA itself OR the OApp's registered delegate to clear an inbound payload. The framework's CPI wrapper, however, presents an API that strongly implies only the OApp PDA can be the signer:

```
RUST
1 /// # Accounts validation
2 /// - `accounts[0]` must equal the caller-supplied `endpoint_program` parameter.
3 /// - `accounts[1]` must match the provided `receiver` key (the OApp PDA).
4 pub fn clear(
5     endpoint_program: Pubkey,
6     receiver: Pubkey,
7     accounts: &[AccountInfo],
8     seeds: &[u8],
9     params: ClearParams,
10 ) -> Result<u8; 32> {
11     validate(accounts, endpoint_program, CLEAR_MIN_ACCOUNTS_LEN)?;
12     require_keys_eq!(accounts[1].key(), receiver, ErrorCode::ConstraintAddress);
13     ...
14 }
```

The parameter is named `receiver` and the docstring labels it "(the OApp PDA)". A delegate-clears flow IS still reachable through this wrapper because the caller controls what they pass as the `receiver` argument (the wrapper only checks that `accounts[1]` matches whatever the caller claims, not that the value actually equals the OApp address), but a developer reading the wrapper signature and docstring has no way to know that.

RECOMMENDATION

Consider updating the wrapper's docstring to make the underlying endpoint behavior explicit, note that the endpoint accepts either the OApp PDA or the registered delegate as the signer, that `accounts[1]` must be whichever signer the caller intends to use, and that `params.receiver` must always be the OApp address regardless of who signs.

RESOLUTION

LayerZero Team - Resolved.

INFO R2-I-03

RESOLVED

Stale OAppInfo Documentation After TLV Migration.

CATEGORY Documentation

LOCATION apps/oapp-app/contracts/solana/macros/src/state_types.rs#L6, apps/oapp-app/contracts/solana/docs/oapp-macro.md#L105

REPO Round 2 - Main Review

DESCRIPTION

The repository's OAppInfo documentation is inconsistent with the current implementation. While the implementation has moved to a raw SPL TLV-backed OAppInfo account with OAppBase as the framework entry and app-specific metadata appended as sibling TLV entries, older documentation still describes OAppInfo as a monolithic versioned PDA containing `InstructionVersions` plus an `extra_info: Vec<u8>` extension blob. This older model is still described in apps/oapp-app/contracts/solana/README.md#L96-L97.

The `#[oapp]` macro documentation is stale in the same area. apps/oapp-app/contracts/solana/docs/oapp-macro.md#L105 still states that the macro prelude auto-generates an OAppInfo account struct, but the current macro emits only `OAppPeer` and `EnforcedOptions` in apps/oapp-app/contracts/solana/macros/src/oapp.rs#L67-L69. In the current design, `oapp_info` must be declared manually as `UncheckedAccount<'info>`, sized explicitly for TLV entries, and initialized with `oapp::init_oapp_base!` plus any app-specific `init_tlv_entry(...)` calls. Some internal and example comments still reinforce the old model as well.

RECOMMENDATION

Update all stale `OAppInfo` references to consistently describe the current manual `UncheckedAccount<'info>` plus TLV-entry model, and remove outdated references to auto-generated `OAppInfo`, `extra_info`, `init_oapp_info!`, and `OAppInfoEntry`.

RESOLUTION

LayerZero Team - Resolved.

INFO R2-I-04

RESOLVED

anchor-trait README stale on wrapper_defs.

CATEGORY Documentation

LOCATION contracts/common/framework/solana/anchor-trait/README.md#L206

REPO Round 2 - Main Review

DESCRIPTION

The `assembly.rs` change in the latest sync moved `wrapper_defs` inside the generated module and updated the inline ASCII diagram in the source doc-comment to reflect the new layout. The "Assembly output structure" diagram in `anchor-trait/README.md` was not updated and still shows `wrapper_defs` as a top-level emission alongside `prelude` and `default_impls`, which contradicts the current code.

The README still says:

RUST

```
1  <prelude>
2  <wrapper_defs>      ← shown outside the module
3  <default_impls>
4  ...
5  [vis] mod <name> { ... }
```

The actual layout, per `assembly.rs` :

RUST

```
1  prelude
2  default_impls
3  ...
4  mod name {
5      filtered_items
6      wrapper_defs      ← now emitted inside, so handlers can reach
7                          //   the user's nested same-module helpers
8      entrypoints
9  }
```

RECOMMENDATION

Mirror the updated diagram from the `assembly.rs` doc-comment into the README so the two stay aligned.

RESOLUTION

LayerZero Team - Resolved.

INFO R2-I-05

RESOLVED

Consider Size Check In `update_tlv_entry`.

CATEGORY Validation LOCATION apps/oapp-app/contracts/solana/src/oapp_info.rs#L189-L200

REPO Round 2 - Main Review

DESCRIPTION

`update_tlv_entry` is only safe when the new value has the same serialized length as the existing TLV entry. The code comments document this and recommend using it for fixed-size entries, but the function itself does not enforce that invariant.

Testing confirmed that misuse has non-obvious failure modes. If a shorter variable-length value is written, the update can succeed while leaving stale trailing bytes in the slot and preserving the original TLV length. If a longer value is written, the update fails, but only after partially overwriting the existing bytes, which can leave the TLV entry corrupted and no longer decodable.

This does not appear to impact the current in-scope metadata structs, which are effectively fixed-size, but it remains an integration footgun for downstream developers.

RECOMMENDATION

Consider rejecting updates when the new serialized length differs from the existing allocated length.

RESOLUTION

LayerZero Team - Resolved.

LOW R2-L-01

ACKNOWLEDGED

OAppInfo Can Reference Missing TLV Entry.

CATEGORY Validation LOCATION apps/oapp-app/contracts/solana/src/oapp_info.rs#L246

REPO Round 2 - Main Review

DESCRIPTION

The TLV-based OAppInfo model relies on discriminator fields to describe which sibling TLV entry should exist next, but that relationship is not validated during initialization. In particular, `init_oapp_base!` writes `OAppBase.app_discriminator`, while the actual sibling entry is written separately through `init_tlv_entry`. Nothing in this flow requires the referenced TLV entry to be present before initialization succeeds.

As a result, a malformed or incomplete initializer can leave `OAppInfo` in an internally inconsistent state where the base entry advertises an app-specific discriminator, but no matching sibling TLV entry exists in the account. This does not appear to create a direct authorization or fund-loss issue, but it can break SDK discovery, off-chain parsing, and transaction construction for application-specific flows that rely on the discriminator chain being valid.

RECOMMENDATION

Add a validation or higher-level initialization helper that ensures any non-zero `app_discriminator` in `OAppBase` corresponds to a sibling TLV entry before initialization succeeds.

RESOLUTION

LayerZero Team - Acknowledged. The generic primitive was kept unchanged, with applications responsible for initializing the full TLV chain.

INFO R2-I-06

RESOLVED

Incorrect composer docs link.

CATEGORY Documentation

LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/composer/lz_compose_types_info.rs#L15

REPO Round 2 - Main Review

DESCRIPTION

The composer macro generator for `lz_compose_types_info()` links to the Solana OApp receive-types documentation at https://docs.layerzero.network/v2/developers/solana/oapp/overview#how-lz_receive_types_v2-works. This file documents composer-specific discovery using `LzComposeParams`, `LzComposeTypesV2Accounts`, and `LZ_COMPOSE_TYPES_SEED`, so the current link sends implementers to the unrelated `lz_receive_types_v2()` flow. This is a documentation-only issue and does not affect generated code or runtime behavior.

RECOMMENDATION

Replace the `See` link with the composer-specific documentation URL:
https://docs.layerzero.network/v2/developers/solana/composer/overview#lz_compose_types_info.

RESOLUTION

LayerZero Team - Resolved.

INFO R2-I-07

RESOLVED

Private repo link in source comment.

CATEGORY Documentation

LOCATION apps/oapp-app/contracts/solana/src/lib.rs#L32

REPO Round 2 - Main Review

DESCRIPTION

The comment immediately before `declare_program!()` links to `https://github.com/LayerZero-Labs/monorepo-internal/...`, which appears to point to a private repository or PR discussion. This does not affect protocol behavior, but external reviewers may be unable to inspect the linked context for why `idls/endpoint.json` is mirrored in this package.

RECOMMENDATION

Be aware that the source includes a reference to private development context. If this source is intended for broader public consumption, consider replacing the private URL with a short public note explaining why `idls/endpoint.json` is mirrored before `declare_program!()`.

RESOLUTION

LayerZero Team - Resolved.

Incomplete package files list.

CATEGORY Configuration LOCATION apps/oapp-app/contracts/solana/package.json#L7-L14

REPO Round 2 - Main Review

DESCRIPTION

The npm package whitelist in `package.json` includes the entire `macros` directory while omitting `idls`. Including `macros` publishes the `macros/examples` tree even though consumers only need `macros/src` and `macros/Cargo.toml`. The examples are also large, which can materially increase package size. More importantly, the package now uses `declare_program!(endpoint)`, which reads `idls/endpoint.json` at compile time. If the package is installed from the published artifact without `idls`, consumers may fail to compile with a missing `idls` directory or accidentally resolve an unrelated ancestor `idls/endpoint.json`.

RECOMMENDATION

Keep the package whitelist limited to compile-time inputs and runtime source

```
JSON
1  "files": [
2    "src",
3    "idls",
4    "macros/src",
5    "macros/Cargo.toml",
6    "Cargo.toml",
7    "Anchor.toml",
8    "rust-toolchain.toml",
9    "rustfmt.toml"
10 ]
```

Do not include `macros` wholesale or build artifacts such as `macros/target`; include `macros/examples` only if examples are intentionally shipped.

RESOLUTION

LayerZero Team - Resolved.

INFO R2-I-09

RESOLVED

register_oapp Prerequisite Documentation Gap.

CATEGORY Documentation

LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/accept_default_admin_transfer.rs#L16-L19

REPO Round 2 - Main Review

DESCRIPTION

The `#[oapp]` macro unconditionally injects an `accept_default_admin_transfer` override that calls `set_delegate`, which requires the OAppRegistry PDA to have been created via `register_oapp`. This prerequisite is documented in the macro implementation comment in `accept_default_admin_transfer.rs`

TEXT

```
1  //! **`register_oapp` is a mandatory call in the OApp's `initialize` handler.**
2  //! Omitting it means `accept_default_admin_transfer` will always fail once a
3  //! default-admin transfer is initiated, permanently bricking the admin transfer
4  //! flow.
```

However, this prerequisite is not surfaced alongside the override description in the main user-facing docs in `README` and `oapp-macro.md`. The bundled counter example also omits `register_oapp` in `counter/src/lib.rs`, which can reinforce the gap for integrators reading the example first.

RECOMMENDATION

Document the `register_oapp` requirement in the main `#[oapp]` docs and examples wherever the generated `accept_default_admin_transfer` override is described.

RESOLUTION

LayerZero Team - Resolved.

INFO R2-I-10

RESOLVED

OAppInfo init accepts nonzero TLV tail.

CATEGORY Validation LOCATION apps/oapp-app/contracts/solana/src/oapp_info.rs#L183-L185, #L254

REPO Round 2 - Main Review

DESCRIPTION

`init_oapp_base!()` initializes the framework `OAppBase` TLV entry through `init_tlv_entry()`, which directly calls `TlvStateMut::unpack()` and then `alloc_and_pack_variable_len_entry()`. The SPL TLV parser treats the first uninitialized discriminator as the end of initialized TLV data, so nonzero bytes after that point are not rejected before the new entry is written.

If `init_oapp_base!()` is called on an `OAppInfo` account whose data is not freshly zero-filled, only the byte range occupied by the new `OAppBase` record is overwritten. Any remaining nonzero tail bytes are preserved. Those bytes may later cause parsing failures or, if they happen to form valid TLV records, be interpreted as unintended sibling entries by SDKs or other readers.

This is not expected in the normal `init` flow because newly allocated Solana account data is zero-filled, but the framework helper itself does not document or validate this assumption.

RECOMMENDATION

Either acknowledge and document that `init_oapp_base!()` must only be called on a freshly initialized, zero-filled `OAppInfo` account, or consider adding additional validation before `OAppBase` initialization to reject nonzero bytes after the first uninitialized TLV slot.

RESOLUTION

LayerZero Team - Resolved.

INFO R2-I-11

RESOLVED

Program attr filtering should be documented.

CATEGORY Documentation

LOCATION contracts/common/framework/solana/anchor-trait/src/assembly.rs#L81-L82

REPO Round 2 - Main Review

DESCRIPTION

`Assembly::assemble()` removes any module attribute whose final path segment is `program` before partitioning attributes into inner and outer attributes. This supports re-emitting a single managed Anchor `#[program]` attribute, including deduplicating qualified forms such as `#[anchor_lang::prelude::program]`.

This behavior can also remove other attributes whose path ends in `program`, even if they are not Anchor's `#[program]` attribute. Because filtering happens before the `AttrStyle::Inner` partition, it also applies to inner attributes. Anchor's `#[program]` attribute is an outer procedural macro on the program module, while `#![program]` as an inner macro attribute is not a stable/usable Anchor program form.

This is not necessarily incorrect if the framework intentionally reserves any attribute path ending in `program`, but that convention is not currently explicit to consumers.

RECOMMENDATION

Document that `anchor-trait` treats outer module attributes ending in `program` as reserved for the managed Anchor `#[program]` emission and may remove them during assembly. Also consider applying the `program` filtering only after partitioning attributes, so inner attributes are preserved while outer Anchor `#[program]` attributes are still deduplicated.

RESOLUTION

LayerZero Team - Resolved.

INFO R2-I-12

RESOLVED

RoleType derive can use nonzero default.

CATEGORY Validation LOCATION contracts/common/utils/solana/rbac/macros/src/role_type.rs#L30-L46

REPO Round 2 - Main Review

DESCRIPTION

`role_type_derive_impl()` only emits the discriminant-zero assertion when it finds a `#[default]` variant. If no `#[default]` variant exists, `default_assert` is empty and the macro still emits `impl ::rbac::traits::RoleType for #name {}`. Since `RoleType` requires `Default` and `default_admin_role()` returns `Self::default()`, a consumer can manually implement `Default` so that `#[derive(RoleType)]` uses a nonzero default-admin discriminant. In that case, `seed()` derives the default-admin role byte from the nonzero discriminant, while the docs and SDK compatibility guidance expect the default-admin PDA seed to be `[0]`.

RECOMMENDATION

Either be aware of and document that `#[derive(RoleType)]` only enforces the zero-discriminant invariant when a `#[default]` variant is present, or change `role_type_derive_impl()` to require a `#[default]` variant and reject derives where it is missing.

`contracts/common/utils/solana/rbac/src/traits.rs#L65-L67` explains how devs may still want to use different default discriminant than 0, but it says `implement this method yourself` which points to `default_admin_role`. Adjust the comment if the intention is to let the dev have different discriminant than 0 by using the `impl Default for ...` syntax.

RESOLUTION

LayerZero Team - Resolved.

INFO **R3-I-01**

ACKNOWLEDGED

Composer codegen doc points to the wrong file.

CATEGORY Documentation

LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/composer/lz_compose_types_info.rs#L16

REPO Round 3 - Remediation Review

DESCRIPTION

The doc comment on the composer macro generator at `apps/oapp-app/contracts/solana/macros/src/instructions/composer/lz_compose_types_info.rs` points to the wrong documentation file

RUST

```
1  //! This follows the same V2 account-discovery model as `lz_receive_types_info`.
2  //! See `docs/lz-receive-types.md#lzcomposetypes-v2-differences` for the
3  //! compose-specific differences.
```

The comment promises "the compose-specific differences" but links to `docs/lz-receive-types.md`, which contains no such section. Those differences live in `docs/lz-compose-types.md` under the heading `### Differences from LzReceiveTypes`. This is a documentation-only issue and does not affect generated code or runtime behavior.

RECOMMENDATION

Repoint the reference to the correct file

RUST

```
1  //! See `docs/lz-compose-types.md` for the compose-specific differences.
```

RESOLUTION

LayerZero Team - Acknowledged.

INFO R3-I-02

ACKNOWLEDGED

Typo in CallerNotPendingAdmin doc comment.

CATEGORY Documentation

LOCATION contracts/common/utils/solana/rbac/src/errors.rs#L7

REPO Round 3 - Remediation Review

DESCRIPTION

The doc comment on the `CallerNotPendingAdmin` error variant at `contracts/common/utils/solana/rbac/src/errors.rs` contains a typo and incorrect terminology

RUST

```
1  /// The caller to accept the default admin transfer neds to be the pending default owner
2  #[msg("Caller is not the pending admin")]
3  CallerNotPendingAdmin,
```

`neds` should be `needs`. Additionally, the comment refers to the actor as the pending default "owner", but the RBAC module models access control in terms of admins, not owners, the variant is `CallerNotPendingAdmin`, its message is `"Caller is not the pending admin"`, and the staged value lives in `pending_default_admin`. This is a documentation-only issue and does not affect generated code or runtime behavior.

RECOMMENDATION

Fix the typo and align the wording with the rest of the module

RUST

```
1  /// The caller of accept_default_admin_transfer must be the pending default admin.
```

RESOLUTION

LayerZero Team - Acknowledged.

INFO R3-I-03

ACKNOWLEDGED

Broken intra-doc link to CodegenContext.

CATEGORY Best Practices LOCATION contracts/common/framework/solana/anchor-trait/src/lib.rs#L63

REPO Round 3 - Remediation Review

DESCRIPTION

The doc comment on the `declare_instructions!` macro at `contracts/common/framework/solana/anchor-trait/src/lib.rs:63` uses rustdoc intra-doc-link syntax (square brackets) for `CodegenContext`

RUST

```
1  /// Appending `(ctx)` after `mod_name` routes to `mod_name::spec(&ctx)`  
2  /// with the [CodegenContext] reference; omitting it calls `spec()` with
```

The `[ ... ]` form instructs rustdoc to resolve `CodegenContext` to a real item and link to it. No such item exists inside the `anchor-trait` crate, `CodegenContext` is only a placeholder name that downstream callers choose for their own context struct (the macro's example passes it via `context = CodegenContext;`, and the associated type is referenced elsewhere as the inert code-span `CodegenContext` at line 42). As a result `cargo doc` emits a `rustdoc::broken_intra_doc_links` diagnostic:

TEXT

```
1  warning: unresolved link to `CodegenContext`  
2    → src/lib.rs:63:16  
3    |  
4  63 | /// with the [CodegenContext] reference; omitting it calls `spec()` with  
5    |                                     ^^^^^^^^^^^^^^^^^ no item named `CodegenContext` in scope
```

By default this is a warning, but it becomes a hard `error: could not document anchor-trait` (non-zero exit) under any deny escalation.

RECOMMENDATION

Drop the brackets so the name is inert text, matching the existing inert usage at line 42

RUST

```
1  /// with the `CodegenContext` reference; omitting it calls `spec()` with
```

RESOLUTION

LayerZero Team - Acknowledged.

INFO R3-I-04

ACKNOWLEDGED

Misleading clear documentation.

CATEGORY Documentation LOCATION apps/oapp-app/contracts/solana/src/endpoint_cpi.rs#L271-L274

REPO Round 3 - Remediation Review

DESCRIPTION

The following documentation was added to `endpoint_cpi::clear()` in response to I-02

```
TEXT
1  /// - `accounts[1]` is the `signer` - pinned to `receiver` (the OApp PDA), which must also sign via
2  ///   `seeds`. The Endpoint itself also accepts the registered delegate as `signer`; that path is
3  ///   intentionally not exposed through this wrapper. To clear as the delegate, CPI to
4  ///   `endpoint::cpi::clear` directly.
```

It says the wrapper doesn't expose the path that allows using delegate as a signer, but that is not true. There are actually 2 receivers. The first is the parameter the `clear` is accepting - the one that's validated to be equal to `accounts[1]` and is passed as signer. And the second is `params.receiver`. This second receiver will be the `OApp PDA`, while the first one can be either the PDA or the delegate. If the user passes the delegate as `receiver`, the endpoint would accept it.

RECOMMENDATION

Either change the documentation to make it clear both paths are accessible or compare `receiver` against `params.receiver` if the intention is to truly disable the delegate path.

RESOLUTION

LayerZero Team - Acknowledged.

INFO R4-I-01

ACKNOWLEDGED

Stale OAppBase doc snippet in oapp-info.

CATEGORY Documentation LOCATION apps/oapp-app/contracts/solana/src/oapp_info.rs REPO Round 4 - Remediation Review

DESCRIPTION

The embedded `OAppBase / IdlVersion` code snippet in `docs/oapp-info.md` does not match the actual struct in `src/oapp_info.rs`. The doc was not synced when the struct was written, so a reader copying or trusting the snippet sees the wrong definition.

Two mismatches:

1. The `schema_version` doc comment carries an extra sentence the real code does not have.

Doc (`docs/oapp-info.md`):

RUST

```
1  /// Borsh schema version of this entry. Bumps when fields are added,
2  /// removed, reordered, or change type. The SDK reads this first to
3  /// select the correct parser for the remaining bytes.
4  pub schema_version: u8,
```

Real code (`src/oapp_info.rs`):

RUST

```
1  /// Borsh schema version of this entry. The SDK reads this first to select the correct parser
2  /// for the remaining bytes.
3  pub schema_version: u8,
```

1. The `IdlVersion` derive list is missing `Debug`.

Doc (`docs/oapp-info.md`):

RUST

```
1  #[derive(Clone, Copy, AnchorSerialize, AnchorDeserialize, InitSpace, PartialEq, Eq)]
2  pub struct IdlVersion {
3      pub major: u8,
4      pub minor: u8,
5  }
```

Real code (`src/oapp_info.rs`):

RUST

```
1  #[derive(Clone, Copy, AnchorSerialize, AnchorDeserialize, InitSpace, PartialEq, Eq, Debug)]
2  pub struct IdlVersion {
3      pub major: u8,
4      pub minor: u8,
5  }
```

RECOMMENDATION

For #1, consider updating the code to reflect the docs. For #2, update the docs and add the missing `Debug`.

RESOLUTION

LayerZero Team - Acknowledged.

INFO R4-I-02

RESOLVED

OAppInfo v1 Can Allow Misleading Discovery.

CATEGORY Documentation

LOCATION apps/oapp-app/contracts/solana/docs/oapp-info.md

REPO Round 4 - Remediation Review

DESCRIPTION

OAppInfo v1 defines a fixed-template discovery model for provided standard OApp/RBAC instructions. SDKs derive required accounts from the published `idl_version` and predefined instruction templates, and schema v1 does not provide a per-instruction override mechanism for those templates.

At the same time, the macro/framework layer continues to support extending or fully overriding provided standard instructions with custom account contexts. This creates a mismatch at the integration boundary: a program can publish OAppInfo v1 while changing the actual account layout of an instruction such as `set_peer` in a way that the v1 metadata format cannot express. A generic SDK or off-chain integrator relying on the advertised standard template may then construct an invalid transaction even though the program behaves as implemented.

The current documentation describes the individual pieces but does not make this incompatibility explicit enough. `oapp-info.md` states v1 has no override path for standard instruction templates, while `oapp-macro.md` and `anchor-trait` documentation separately describe extending or overriding provided instructions. There is no clear statement that programs publishing OAppInfo v1 for standardized discovery must not extend or fully override the account contexts of SDK-discoverable provided standard instructions.

RECOMMENDATION

Explicitly document that programs publishing OAppInfo v1 must not extend or fully override the accounts of standard provided instructions.

RESOLUTION

LayerZero Team - Resolved.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian.

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.